

Assigment 2

Submission requirements

Your homework will not be graded if your notebook doesn't include output. In other words, **make sure to rerun your notebook before submitting to Gradescope** (Note: if you are using Google Colab: go to Edit > Notebook Settings and uncheck Omit code cell output when saving this notebook, otherwise the output is not printed).

Additional points may be deducted if these requirements are not met:

- Comment your code;
- Each graph should have a title, labels for each axis, and (if needed) a legend. Each graph should be understandable on its own;
- Try and minimize the use of the global namespace (meaning, keep things inside functions).

Import libraries

```
In [70]: import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt
```

Define functions

```
In [71]: def create_1d_data(num_examples, w, b, bound):
    """Create X, Y data with a linear relationship with added noise.

    Args:
        num_examples: number of examples to generate
        w: desired slope
        b: desired intercept
        bound: lower and upper boundary of the data interval

    Returns:
        X and Y with shape (num_examples)
    """
    np.random.seed(4) # consistent random number generation
    X = np.arange(num_examples)
    deltas = np.random.uniform(low=-bound, high=bound, size=X.shape) # added noise
    Y = b + deltas + w * X

    return X, Y
```

Step 1: Data ingestion

Let's start by creating some artificial data using the `create_1d_data()` function defined at the top of the notebook.

```
In [72]: # Define parameters
num_examples = 70
w = 2
b = 1
bound = 1

# Create data
X, Y = create_1d_data(num_examples, w, b, bound)

# Print shapes
print('Printing shape of X:', X.shape)
print('Printing first 10 elements in X:', X[:10])
print('\nPrinting shape of Y:', Y.shape)
print('Printing first 10 elements in Y:', Y[:10])
```

Printing shape of X: (70,)

Printing first 10 elements in X: [0 1 2 3 4 5 6 7 8 9]

Printing shape of Y: (70,)

Printing first 10 elements in Y: [1.93405968 3.0944645 5.94536872 7.42963199
9.39545765 10.43217899
13.95254891 14.01246051 16.50596472 18.86958306]

Step 2: Data Preprocessing

Our objectives here involve adding one more feature to X and creating data partitions.

Exercise 1: Adding features (10 points)

1. Add a column of 1s to X (this will serve as an intercept or "bias" in our modeling task later on). Note: do not create a new array, just concatenate with the current values;
2. Print the shape of the array.

```
In [73]: # YOUR CODE HERE

# 1. add column of 1s to X
n = X.shape[0]
# first, must reshape X to have 2 dimensions in order to add a column of ones.
X = np.concatenate((X.reshape(n, 1), np.ones((n, 1))), axis=1)

# 2. print shape of array
print(X.shape)
```

(70, 2)

Exercise 2: Data splits (10 points)

Using the `train_test_split()` method available in scikit-learn:

1. Split the (X,Y) data into training and test partitions by setting `test_size=0.2` and `random_state=1234`. All the other arguments of the method are set to default values. Name the resulting arrays `X_train`, `X_test`, `Y_train`, `Y_test`;
2. Print the shape of each array.

```
In [74]: # YOUR CODE HERE
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=1234)
print('X train shape', X_train.shape)
print('X test shape', X_test.shape)
print('Y train shape', Y_train.shape)
print('Y test shape', Y_test.shape)
```

```
X train shape (56, 2)
X test shape (14, 2)
Y train shape (56,)
Y test shape (14,)
```

Step 3: Exploratory data analysis (EDA)

Exercise 3: Plots (10 points)

Create a side-by-side histogram for the values in `Y_train` and `X_train`. Make sure to include axes name and titles for each subplot.

```
In [ ]: # YOUR CODE HERE

# Create a figure with 1 row and 2 columns
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 5))

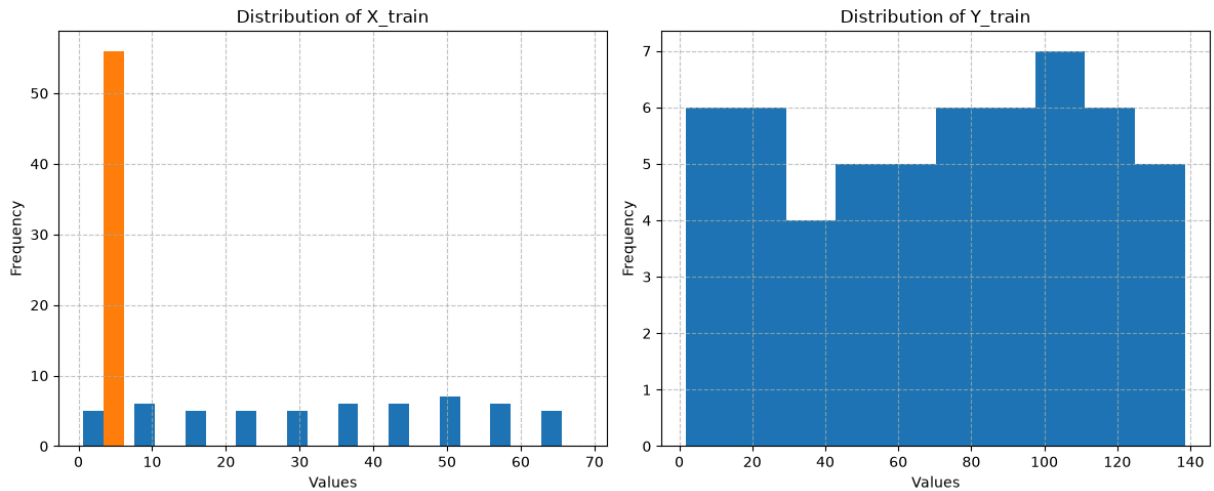
# Plot the first histogram (X_train) on the left axis
axes[0].hist(X_train, bins=10)
axes[0].set_title('Distribution of X_train')
axes[0].set_xlabel('Values')
axes[0].set_ylabel('Frequency')
axes[0].grid(True, linestyle='--', alpha=0.7)

# Plot the second histogram (Y_train) on the right axis
axes[1].hist(Y_train, bins=10)
axes[1].set_title('Distribution of Y_train')
axes[1].set_xlabel('Values')
axes[1].set_ylabel('Frequency')
axes[1].grid(True, linestyle='--', alpha=0.7)

# Adjust spacing
```

```
plt.tight_layout()

# Display the plots
plt.show()
```



Step 4: Modeling

Let's start by considering the two models proposed in Assignment 1:

1. $M_1(x) = 5 + x$
2. $M_2(x) = 1 + 2x$

Exercise 4: Practice with Parameters (10 points)

Perform the following computations:

1. Use matrix multiplication (`np.dot`) to create M_1 and M_2 (as previously defined) to produce vectors of predictions using the `X_train` data. Call these predictions `M1_hat_train`, `M2_hat_train`. Hint: the "learned" parameters are already provided to you;
2. Print the shapes of the predictions to verify that they match the shape of `Y_train`.

```
In [77]: X_train.dot(np.array([1,0]))
```

```
Out[77]: array([50., 65., 21., 52., 45., 40., 13., 67., 27., 58., 10., 7., 31.,
        14., 17., 56., 32., 35., 42., 1., 64., 19., 2., 59., 51., 62.,
         0., 11., 39., 3., 68., 34., 37., 12., 55., 69., 9., 16., 5.,
        28., 48., 54., 61., 44., 60., 43., 30., 26., 41., 23., 49., 15.,
        24., 53., 38., 47.])
```

```
In [78]: # YOUR CODE HERE
```

```
# 1. M1 = 5 + X, M2 = 1 + 2X
M1_hat_train = 5 + X_train.dot(np.array([1,0]))
M2_hat_train = 1 + 2 * np.dot(X_train, np.array([1,0]))
```

```
# 2. print shapes of preds
print('shapes of preds')
print('M1 preds shape', M1_hat_train.shape)
print('M2 preds shape', M2_hat_train.shape)
```

```
shapes of preds
M1 preds shape (56,)
M2 preds shape (56,)
```

Exercise 5: Learn parameters with Gradient Descent (20 points)

1. Fill in the **NotImplemented** parts of the **gradient_descent()** function below. Hint: refer to **02 Linear Regression_helper.ipynb** file ;
2. Run this function with our artificial (X_train, Y_train) data . Set learning_rate = .0002 and num_epochs = 5. Print out the weights and loss after each epoch.
3. Generate a plot with the loss values on the y-axis and the epoch number on the x-axis for visualization. Make sure to include axes name and title.

```
In [79]: num_epochs = 5
for i in range(num_epochs):
    print(i)
```

```
0
1
2
3
4
```

```
In [80]: def gradient_descent(inputs, outputs, learning_rate, num_epochs):
    """Apply the gradient descent algorithm to learn linear regression.

    Args:
        inputs: A 2-D array where each column is an input feature and each
            row is a training example.
        outputs: A 1-D array containing the real-valued
            label corresponding to the input data in the same row.
        learning_rate: The learning rate to use for updates.
        num_epochs: The number of passes through the full training data.

    Returns:
        weights: A 2-D array with the learned weights after each training epoch.
        losses: A 1-D array with the loss after each epoch.
    """
    # m = number of examples, n = number of features
    m, n = inputs.shape

    # We'll use a vector of size n to store the learned weights and initialize
    # all weights to 1.
    W = np.ones(n)

    # Keep track of the training loss and weights after each step.
```

```

losses = []
weights = []

for epoch in range(num_epochs):
    # Append the old weights to the weights list to keep track of them.
    weights.append(W)

    # Evaluate the current predictions for the training examples given
    # the current estimate of W.
    predictions = inputs.dot(W.T)

    # Find the difference between the predictions and the actual target
    # values.
    diff = predictions - outputs

    # In standard linear regression, we want to minimize the sum of squared
    # differences. Compute the mean squared error Loss. Don't bother with the
    # 1/2 scaling factor here.
    loss = (1/m) * np.mean(np.pow(diff, 2))

    # Append the Loss to the losses list to keep a track of it.
    losses.append(loss)

    # Compute the gradient with respect to the Loss.
    # [Formula (4) in the Gradient Descent Implementation]
    gradient = (2 / inputs.shape[0]) * inputs.T.dot(predictions - outputs)

    # Update weights, scaling the gradient by the Learning rate.
    W = W - learning_rate * gradient

    print(f'Epoch {epoch}')
    print('Weights', weights)
    print('Loss', losses)

    # Append the final weights after the loop
    weights.append(W)

return np.array(weights), np.array(losses)

```

In [81]: # YOUR CODE HERE

```

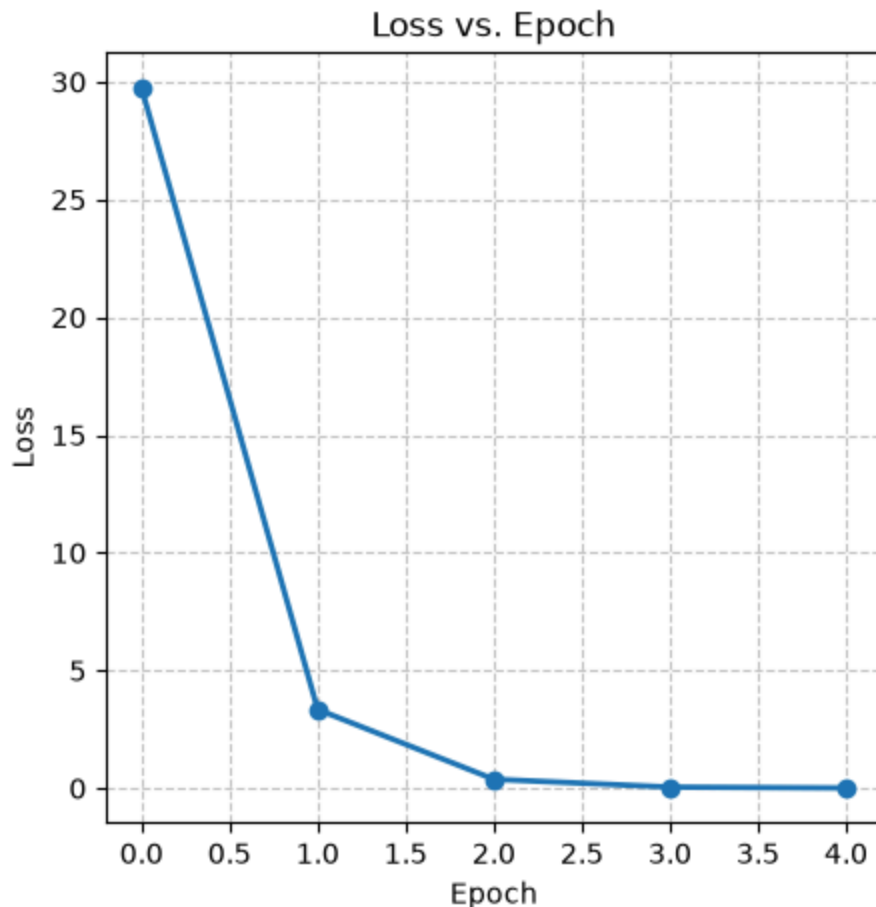
weights, losses = gradient_descent(X_train, Y_train, learning_rate=0.0002, num_epochs=1000)

# Loss curve
epochs = np.arange(len(losses))

plt.figure(figsize=(5, 5))
plt.plot(epochs, losses, marker='o', color='tab:blue', linewidth=2)
plt.title('Loss vs. Epoch')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```

```
Epoch 0
Weights [array([1., 1.])]
Loss [np.float64(29.738690372585868)]
Epoch 1
Weights [array([1., 1.]), array([1.66525772, 1.01418788])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387)]
Epoch 2
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.01895194])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3811508546356395)]
Epoch 3
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.01895194]), array([1.96299757, 1.02055715])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3811508546356395), np.float64(0.04796807650761877)]
Epoch 4
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.01895194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465)]
```



Step 5: Hyperparameter tuning

Hyperparameter tuning plays an important role in optimizing ML models. One systematically adjusts hyperparameters to find the right balance between model complexity and generalization, ultimately leading to better predictive performance and model effectiveness.

Note that hyperparameter tuning is typically performed on **a separate validation dataset**. However, for the purposes of this assignment and based on the ML knowledge you've acquired thus far, we will perform hyperparameter tuning directly on the training data.

Exercise 6: Tuning hyperparameters (10 points)

Answer the following questions:

1. Do you think it would be beneficial to extend the model training in Exercise 5 by increasing the learning rate to 0.02? Justify your answer;
2. Restoring to the previous learning_rate = .0002, would you consider extending the model training in Exercise 5 by increasing the number of epochs to 10? Justify your answer.

Note: to support your answers, we recommend the following actions:

- create side-by-side subplots to show the loss at each epoch (make sure to include axes name and title), and
- print the loss at the last epoch under the following scenarios:
 - learning_rate=.0002, num_epochs=5;
 - learning_rate=.02, num_epochs=5;
 - learning_rate=.0002, num_epochs=10.

Answers

1. No it would not be beneficial to model training to increase the learning rate to 0.02, this would be an increase to the learning rate by a factor of 100. As we can see in the loss curve, the loss seems to have already converged by epoch 3. Recall that linear regression's loss function is convex, so if we were to increase the learning rate to this extent, we would likely make inefficient parameter updates and the loss would bounce around rather than monotonically decrease.
2. Yes I'd consider extending the training to 10 epochs to get a marginally smaller loss, but as noted in my above answer to 1, there probably wouldn't be much of a point considering we seem to have converged with the 5 epochs of training. In fact, it might be counterintuitive to extend training to more epochs because there could be risk of overfitting.

```
In [82]: # YOUR CODE HERE
# Run the three scenarios
loss_0002_5, loss_02_5, loss_0002_10 = [], [], []
```

```

_, losses_a = gradient_descent(X_train, Y_train, learning_rate=0.0002, num_epochs=5)
_, losses_b = gradient_descent(X_train, Y_train, learning_rate=0.02, num_epochs=5)
_, losses_c = gradient_descent(X_train, Y_train, learning_rate=0.0002, num_epochs=10)

# Print Last Loss for each case
print("learning_rate=0.0002, num_epochs=5 -> final loss:", losses_a[-1])
print("learning_rate=0.02, num_epochs=5 -> final loss:", losses_b[-1])
print("learning_rate=0.0002, num_epochs=10 -> final loss:", losses_c[-1])

# Create side-by-side subplots
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Scenario 1
epochs_a = np.arange(len(losses_a))
axes[0].plot(epochs_a, losses_a, marker='o', color='tab:blue')
axes[0].set_title('lr = 0.0002, epochs = 5')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].grid(True, linestyle='--', alpha=0.7)

# Scenario 2
epochs_b = np.arange(len(losses_b))
axes[1].plot(epochs_b, losses_b, marker='o', color='tab:red')
axes[1].set_title('lr = 0.02, epochs = 5')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].grid(True, linestyle='--', alpha=0.7)

# Scenario 3
epochs_c = np.arange(len(losses_c))
axes[2].plot(epochs_c, losses_c, marker='o', color='tab:green')
axes[2].set_title('lr = 0.0002, epochs = 10')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Loss')
axes[2].grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

```

```

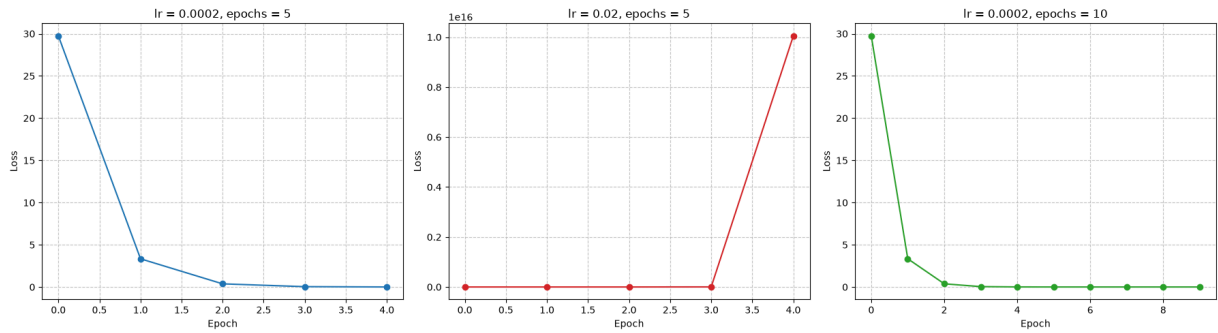
Epoch 0
Weights [array([1., 1.])]
Loss [np.float64(29.738690372585868)]
Epoch 1
Weights [array([1., 1.]), array([1.66525772, 1.01418788])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387)]
Epoch 2
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395)]
Epoch 3
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877)]
Epoch 4
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465)]
Epoch 0
Weights [array([1., 1.])]
Loss [np.float64(29.738690372585868)]
Epoch 1
Weights [array([1., 1.]), array([67.5257721 , 2.41878839])]
Loss [np.float64(29.738690372585868), np.float64(127484.32426751219)]
Epoch 2
Weights [array([1., 1.]), array([67.5257721 , 2.41878839]), array([-4288.59461095,
-90.4006607 ])]
Loss [np.float64(29.738690372585868), np.float64(127484.32426751219), np.float64(546
608414.0919319)]
Epoch 3
Weights [array([1., 1.]), array([67.5257721 , 2.41878839]), array([-4288.59461095,
-90.4006607 ]), array([280950.89027965, 5987.51452476])]
Loss [np.float64(29.738690372585868), np.float64(127484.32426751219), np.float64(546
608414.0919319), np.float64(2343666749946.651)]
Epoch 4
Weights [array([1., 1.]), array([67.5257721 , 2.41878839]), array([-4288.59461095,
-90.4006607 ]), array([280950.89027965, 5987.51452476]), array([-18396577.9871651
5, -391995.3005066 ])]
Loss [np.float64(29.738690372585868), np.float64(127484.32426751219), np.float64(546
608414.0919319), np.float64(2343666749946.651), np.float64(1.0048827813945534e+16)]
Epoch 0
Weights [array([1., 1.])]
Loss [np.float64(29.738690372585868)]
Epoch 1
Weights [array([1., 1.]), array([1.66525772, 1.01418788])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387)]
Epoch 2
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395)]
Epoch 3
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189

```

```

5194]), array([1.96299757, 1.02055715]))
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877)]
Epoch 4
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465)]
Epoch 5
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352]), array([1.9
9645053, 1.02129496])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465), n
p.float64(0.006326123993846987)]
Epoch 6
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352]), array([1.9
9645053, 1.02129496]), array([1.9992654 , 1.02136744])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465), n
p.float64(0.006326123993846987), np.float64(0.005853510118291119)]
Epoch 7
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352]), array([1.9
9645053, 1.02129496]), array([1.9992654 , 1.02136744]), array([2.00020876, 1.0214000
3])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465), n
p.float64(0.006326123993846987), np.float64(0.005853510118291119), np.float64(0.0058
00395841239472)]
Epoch 8
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352]), array([1.9
9645053, 1.02129496]), array([1.9992654 , 1.02136744]), array([2.00020876, 1.0214000
3]), array([2.0005248 , 1.02141926])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465), n
p.float64(0.006326123993846987), np.float64(0.005853510118291119), np.float64(0.0058
00395841239472), np.float64(0.005794415671397154)]
Epoch 9
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.0189
5194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352]), array([1.9
9645053, 1.02129496]), array([1.9992654 , 1.02136744]), array([2.00020876, 1.0214000
3]), array([2.0005248 , 1.02141926]), array([2.00063055, 1.021434  ])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3
811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465), n
p.float64(0.006326123993846987), np.float64(0.005853510118291119), np.float64(0.0058
00395841239472), np.float64(0.005794415671397154), np.float64(0.005793731394507902)]
learning_rate=0.0002, num_epochs=5 -> final loss: 0.0105323382624465
learning_rate=0.02, num_epochs=5 -> final loss: 1.0048827813945534e+16
learning_rate=0.0002, num_epochs=10 -> final loss: 0.005793731394507902

```



Exercise 7: Choosing the best model (10 points)

Write down the learned model that you consider most optimal given your answers to the hyperparameter tuning exercise. Call this model $M_3(x)$.

```
In [83]: # YOUR ANSWER HERE

# most optimal learned model is lr = 0.0002, epochs = 5
weights, losses = gradient_descent(X_train, Y_train, learning_rate=0.0002, num_epochs=5)
M3_weights = weights[-1] # weights from last epoch

print(M3_weights.shape)
print(X_train.shape)
M3_hat_train = X_train.dot(M3_weights)
```

```
Epoch 0
Weights [array([1., 1.])]
Loss [np.float64(29.738690372585868)]
Epoch 1
Weights [array([1., 1.]), array([1.66525772, 1.01418788])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387)]
Epoch 2
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.01895194])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3811508546356395)]
Epoch 3
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.01895194]), array([1.96299757, 1.02055715])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3811508546356395), np.float64(0.04796807650761877)]
Epoch 4
Weights [array([1., 1.]), array([1.66525772, 1.01418788]), array([1.88825083, 1.01895194]), array([1.96299757, 1.02055715]), array([1.98805237, 1.02110352])]
Loss [np.float64(29.738690372585868), np.float64(3.3465203492154387), np.float64(0.3811508546356395), np.float64(0.04796807650761877), np.float64(0.0105323382624465)]
(2,)
(56, 2)
```

Step 6: Evaluation and Generalization

To evaluate our models, we assess their performance based on how closely they fit the available data. In other words, we compare the true value y with the predicted value $\hat{y}$ for each x .

Exercise 8: Computing MSE (10 points)

Assess the performance of the three models ($M_1(x)$, $M_2(x)$, $M_3(x)$) by computing the MSE metric on the training dataset. Hint: you can use the `mean_squared_error()` method available in `sklearn.metrics`.

In [84]: `# YOUR CODE HERE`

```
print('M1 train MSE', mean_squared_error(Y_train, M1_hat_train))
print('M2 train MSE', mean_squared_error(Y_train, M2_hat_train))
print('M3 train MSE', mean_squared_error(Y_train, M3_hat_train))
```

```
M1 train MSE 1397.608983658733
M2 train MSE 0.3280557752885262
M3 train MSE 0.35426294365543126
```

Exercise 9: Generalization (10 points)

Which of the three models ($M_1(x)$, $M_2(x)$, $M_3(x)$) demonstrates better generalization ability? Hint: compare training vs. test MSE.

In [85]: `# YOUR CODE HERE`

```
M1_hat_test = 5 + X_test.dot(np.array([1,0]))
M2_hat_test = 1 + 2 * np.dot(X_test, np.array([1,0]))
M3_hat_test = X_test.dot(M3_weights)

print('M1 train MSE', mean_squared_error(Y_train, M1_hat_train))
print('M1 test MSE', mean_squared_error(Y_test, M1_hat_test))
print('M2 train MSE', mean_squared_error(Y_train, M2_hat_train))
print('M2 test MSE', mean_squared_error(Y_test, M2_hat_test))
print('M3 train MSE', mean_squared_error(Y_train, M3_hat_train))
print('M3 test MSE', mean_squared_error(Y_test, M3_hat_test))
```

```
M1 train MSE 1397.608983658733
M1 test MSE 1113.6003911560044
M2 train MSE 0.3280557752885262
M2 test MSE 0.2908006971666422
M3 train MSE 0.35426294365543126
M3 test MSE 0.2945718006278488
```

Answer: It's hard to choose between M2 and M3 because they have such similar test MSEs, M2's is 0.291 and M3's is 0.295. M2 does have a smaller difference between its train and test MSE: 0.038, while M3 has a difference between its train and test MSE of 0.06. You could go with M2 as it has lower test MSE, or you could go with M3 with its better test MSE compared to its train MSE. I think I'd go with M2 personally because it does indeed have a smaller test MSE.

Exercise 10: Additional practice (not graded)

1. How does stochastic gradient descent (SGD) differ from the gradient descent (GD) approach implemented in Exercise 5?
2. Determine the optimal weights for model $M_3(x)$ using SGD. Does SGD converge more rapidly? Explain the reasons for your answer.

In [86]: `# YOUR ANSWER HERE`

In [87]: `# YOUR CODE HERE`